



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AN INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN USING GRAPHS, HEAPS AND PRIORITY QUEUES

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

S. Vinil Kumar (192511226)

T. Varun Tej (192511217)

R.U Sundaresan (192424359)

Under the Supervision

Of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 September

2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply suitable Abstract Data Types for efficient disaster-relief supply management. CO4: Use heaps and priority queues for prioritizing emergency supply requests. CO5: Develop and evaluate an efficient disaster-relief supply chain system under varying loads.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping	SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Efficient management of relief supplies, emergency requests and transportation routes for faster delivery to disaster-affected areas.

B. Assignment Problem / Challenge

A disaster-relief organization must deliver essential supplies such as food, water, medicines, and emergency equipment from warehouses to multiple disaster-affected locations. Each location has different requirements, urgency levels, available routes, and transportation distances.

The challenge is to design an efficient C-based disaster-relief supply chain system that can represent the transportation network using a graph, identify suitable routes between locations, prioritize urgent supply requests using a heap and priority queue, and ensure that critical areas receive supplies before lower-priority locations.

The system should efficiently manage increasing numbers of locations, routes, and relief requests while minimizing delays in emergency supply distribution.

C. Problem Statement

The proposed system is designed to manage the distribution of essential disaster-relief supplies from warehouses to affected areas through an interconnected transportation network. The system represents warehouses, relief centres, hospitals, and affected locations as vertices of a graph, while roads or transportation routes are represented as weighted edges.

Each relief request contains information such as the destination, required supply, quantity, and urgency level. A priority queue implemented using a max heap is used to ensure that the most urgent requests are processed first. Graph-based route analysis is used to identify suitable paths for transporting supplies between locations.

The system should efficiently handle multiple relief requests, varying route distances, limited supplies, and increasing numbers of locations. The student develops a complete C program integrating graphs, heaps, and priority queues, followed by testing and performance evaluation under different numbers of locations and relief requests.

D. Requirements and Constraints

Functional Requirements:

- Store details of warehouses, relief centres, hospitals, and disaster-affected locations.
- Represent the transportation network using a weighted graph.
- Store and process relief requests based on their urgency level.
- Prioritize high-urgency requests using a max heap priority queue.
- Identify suitable routes between supply centres and affected locations.
- Display the order in which relief requests are processed.
- Track the quantity and type of supplies required at each location.
- Generate performance measurements for different numbers of locations and relief requests.

Performance Requirements:

- Graph insertion and adjacency operations should be efficient for large networks.
- Priority queue insertion and deletion: $O(\log n)$.
- Highest-priority request retrieval: $O(1)$.
- Graph traversal using BFS/DFS: $O(V + E)$.
- Support increasing numbers of locations, routes, and relief requests with efficient memory usage.

Technical Constraints:

- Implementation language: C.
- Graph representation: Adjacency list.
- Route information: Weighted edges representing distance or travel time.
- Priority queue: Max Heap.
- Priority value: Disaster urgency score.
- Performance measurement: clock() function.

Security, Reliability and Ethical Constraints

- Relief requests must be processed according to their correct urgency levels.
- The system should not lose or duplicate supply requests during processing.
- Route information must be maintained accurately to avoid incorrect deliveries.
- Disaster-related information should be used only for legitimate relief and emergency-management purposes.

E. Student Work

1. Problem Understanding and Formulation

The problem is to develop an intelligent disaster-relief supply chain system that can efficiently manage the movement of essential supplies from warehouses to disaster-affected locations. Traditional methods of handling relief requests may become inefficient when the number of locations and emergency requests increases.

A graph is used to represent the transportation network, where locations are represented as vertices and connecting roads are represented as weighted edges. A priority queue implemented using a max heap is used to arrange relief requests according to their urgency. This combination allows the system to identify important requests quickly and support efficient relief distribution.

Expected Outcomes

- Efficient representation of the disaster-relief transportation network.
- Faster identification of suitable routes between supply locations and affected areas.
- Immediate identification of the highest-priority relief request.
- Efficient processing of multiple emergency supply requests.
- Performance comparison for increasing numbers of locations, routes, and requests.

Available Data

Field	Example
Source Location	Central Warehouse
Destination	Relief Centre A
Supply Type	Drinking Water
Quantity	500 Units
Urgency Score	95
Route Distance	12 km

Assumptions

- Urgency scores range from 0 to 100.
- A higher urgency score indicates a greater need for immediate relief.
- Locations are represented as vertices in the graph.
- Roads or transportation routes are represented as weighted edges.
- Edge weights represent distance or travel time.
- Relief requests are processed using a max heap priority queue.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
graph	Represents warehouses, relief centres, hospitals and disaster-affected areas as connected locations	BFS/DFS: $O(V + E)$
Adjacency List	Stores the roads and transportation routes between locations efficiently	$O(V + E)$
Weighted Graph	Represents distance or travel time between connected locations	Depends on graph operation
Max Heap	Maintains relief requests according to urgency score	$O(\log n)$ insertion/deletion
Priority Queue	Processes the most urgent disaster-relief request first	$O(1)$ highest-priority retrieval

3.Solution / Design / Methodology

The system represents disaster locations and transportation routes using a weighted graph, while relief requests are organized using a max heap priority queue.

The graph helps identify connected routes, and the priority queue ensures that the most urgent requests are processed first.

The system processes requests sequentially and evaluates its performance under different numbers of locations and relief requests.

Algorithm

1. Initialize the weighted graph and max heap priority queue.
2. Add locations and connecting routes with their distance or travel-time weights.
3. Read each relief request and insert it into the max heap based on urgency score.
4. Select the highest-priority request and determine a suitable route through the graph.
5. Deliver the required supply, remove the processed request, and repeat until all requests are co

Flow Chart

Disaster-Relief Supply Chain System using Graphs, Heaps and Priority Queues

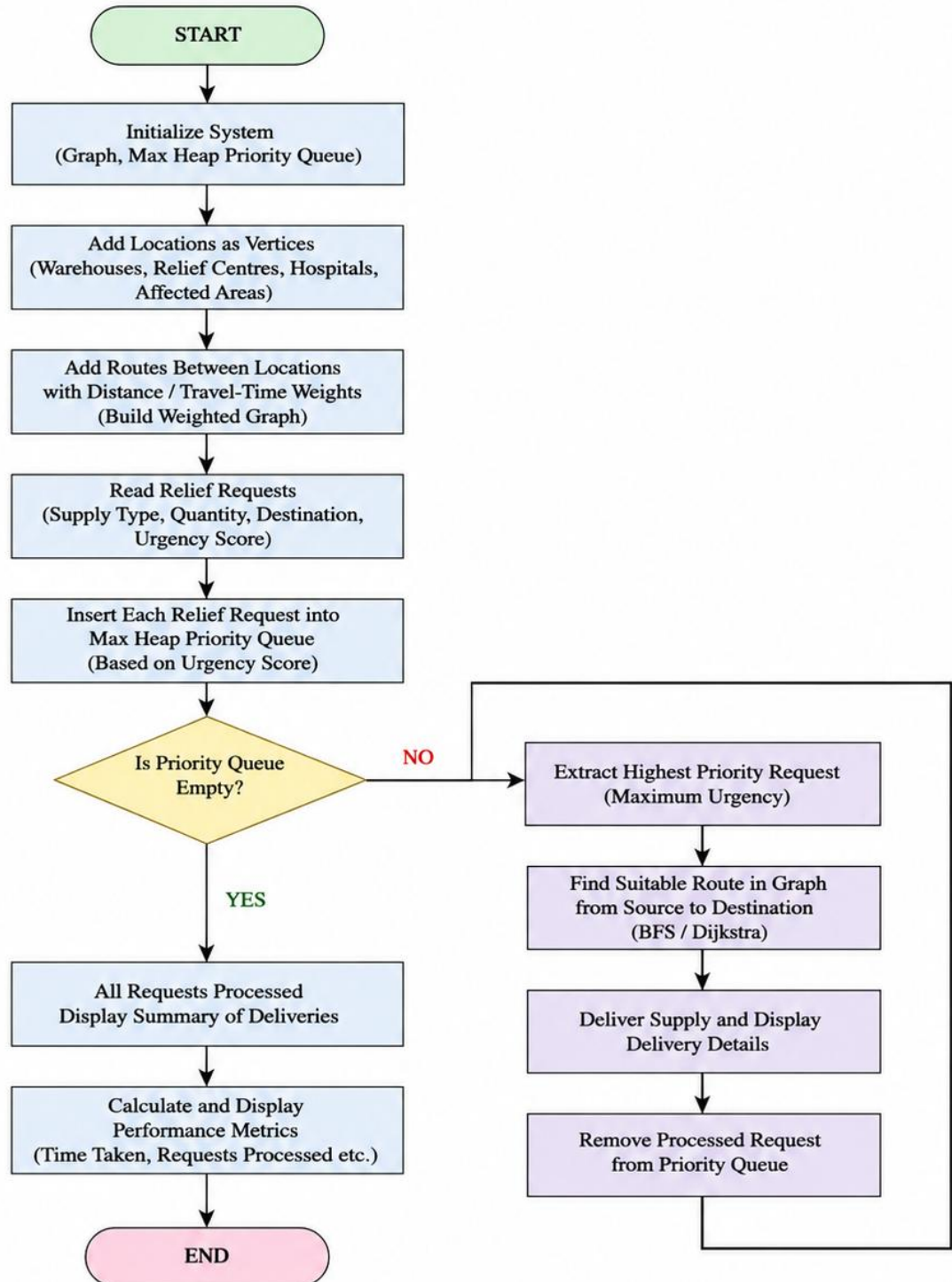


Fig: 1 Flow Chart of disaster relief supply chain system

Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <time.h>

#define MAX_LOCATIONS 20
#define MAX_REQUESTS 100
#define MAX_SUPPLY 1000
#define INF 999999

/* ----- LOCATION / GRAPH ----- */

typedef struct Edge {
    int destination;
    int distance;
    struct Edge *next;
} Edge;

typedef struct {
    char name[50];
    Edge *head;
} Location;

Location graph[MAX_LOCATIONS];
int locationCount = 0;

/* Add a new location */
int addLocation(char name[]) {
    if (locationCount >= MAX_LOCATIONS) {
        printf("Maximum number of locations reached.\n");
        return -1;
    }

    strcpy(graph[locationCount].name, name);
    graph[locationCount].head = NULL;

    return locationCount++;
}

/* Add a route between two locations */
void addRoute(int source, int destination, int distance) {
    Edge *newEdge = (Edge *)malloc(sizeof(Edge));

    newEdge->destination = destination;
    newEdge->distance = distance;
```



```

    newEdge->next = graph[source].head;

    graph[source].head = newEdge;
}

/* Add a bidirectional route */
void addBidirectionalRoute(int source, int destination, int distance) {
    addRoute(source, destination, distance);
    addRoute(destination, source, distance);
}

/* Display graph */
void displayGraph() {
    printf("\n===== DISASTER RELIEF TRANSPORTATION NETWORK =====\n");

    for (int i = 0; i < locationCount; i++) {
        printf("%s -> ", graph[i].name);

        Edge *temp = graph[i].head;

        while (temp != NULL) {
            printf("%s (%d km)",
                graph[temp->destination].name,
                temp->distance);

            if (temp->next != NULL)
                printf(" -> ");

            temp = temp->next;
        }

        printf("\n");
    }
}

/* ----- RELIEF REQUEST ----- */

typedef struct {
    int requestID;
    char source[50];
    char destination[50];
    char supplyType[40];
    int quantity;
    int urgency;
} ReliefRequest;

/* ----- MAX HEAP / PRIORITY QUEUE ----- */

ReliefRequest heap[MAX_REQUESTS];

```

```

int heapSize = 0;

/* Swap two requests */
void swapRequests(ReliefRequest *a, ReliefRequest *b) {
    ReliefRequest temp = *a;
    *a = *b;
    *b = temp;
}

/* Insert request into max heap */
void insertPriorityQueue(ReliefRequest request) {

    if (heapSize >= MAX_REQUESTS) {
        printf("Priority queue is full.\n");
        return;
    }

    int i = heapSize;
    heap[heapSize++] = request;

    while (i > 0) {

        int parent = (i - 1) / 2;

        if (heap[parent].urgency >= heap[i].urgency)
            break;

        swapRequests(&heap[parent], &heap[i]);

        i = parent;
    }
}

/* Remove highest-priority request */
ReliefRequest extractHighestPriority() {

    ReliefRequest result = heap[0];

    heap[0] = heap[--heapSize];

    int i = 0;

    while (1) {

        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int largest = i;

        if (left < heapSize &&

```

```

        heap[left].urgency > heap[largest].urgency)
        largest = left;

    if (right < heapSize &&
        heap[right].urgency > heap[largest].urgency)
        largest = right;

    if (largest == i)
        break;

    swapRequests(&heap[i], &heap[largest]);

    i = largest;
}

return result;
}

/* Display priority queue */
void displayPriorityQueue() {

    printf("\n===== RELIEF REQUEST PRIORITY QUEUE =====\n");

    if (heapSize == 0) {
        printf("No pending requests.\n");
        return;
    }

    for (int i = 0; i < heapSize; i++) {

        printf("Request ID: %d | Destination: %-15s | "
            "Supply: %-15s | Quantity: %d | Urgency: %d\n",
            heap[i].requestID,
            heap[i].destination,
            heap[i].supplyType,
            heap[i].quantity,
            heap[i].urgency);
    }
}

/* ----- DIJKSTRA'S ALGORITHM ----- */

void findShortestRoute(int source, int destination) {

    int distance[MAX_LOCATIONS];
    int visited[MAX_LOCATIONS];
    int parent[MAX_LOCATIONS];

    for (int i = 0; i < locationCount; i++) {

```

```

    distance[i] = INF;
    visited[i] = 0;
    parent[i] = -1;
}

distance[source] = 0;

/* Dijkstra */
for (int count = 0; count < locationCount; count++) {

    int minDistance = INF;
    int current = -1;

    for (int i = 0; i < locationCount; i++) {

        if (!visited[i] &&
            distance[i] < minDistance) {

            minDistance = distance[i];
            current = i;
        }
    }

    if (current == -1)
        break;

    visited[current] = 1;

    Edge *temp = graph[current].head;

    while (temp != NULL) {

        int next = temp->destination;

        if (!visited[next] &&
            distance[current] + temp->distance <
            distance[next]) {

            distance[next] =
                distance[current] + temp->distance;

            parent[next] = current;
        }

        temp = temp->next;
    }
}

if (distance[destination] == INF) {

```

```

    printf("No route available from %s to %s.\n",
           graph[source].name,
           graph[destination].name);

    return;
}

/* Construct route */
int path[MAX_LOCATIONS];
int pathLength = 0;

int current = destination;

while (current != -1) {

    path[pathLength++] = current;
    current = parent[current];
}

printf("\nRoute: ");

for (int i = pathLength - 1; i >= 0; i--) {

    printf("%s", graph[path[i]].name);

    if (i != 0)
        printf(" -> ");
}

printf("\nTotal Distance: %d km\n",
       distance[destination]);
}

/* ----- FIND LOCATION ----- */

int findLocation(char name[]) {

    for (int i = 0; i < locationCount; i++) {

        if (strcmp(graph[i].name, name) == 0)
            return i;
    }

    return -1;
}

/* ----- PROCESS REQUESTS ----- */

```

```

void processRequests() {

    printf("\n\n=====\\n");
    printf("        DISASTER RELIEF DELIVERY PROCESS\\n");
    printf("=====\\n");

    while (heapSize > 0) {

        ReliefRequest request =
            extractHighestPriority();

        printf("\n-----\\n");

        printf("Processing Request ID : %d\\n",
            request.requestID);

        printf("Source          : %s\\n",
            request.source);

        printf("Destination       : %s\\n",
            request.destination);

        printf("Supply Type       : %s\\n",
            request.supplyType);

        printf("Quantity          : %d units\\n",
            request.quantity);

        printf("Urgency Score     : %d\\n",
            request.urgency);

        int source =
            findLocation(request.source);

        int destination =
            findLocation(request.destination);

        if (source == -1 || destination == -1) {

            printf("Invalid source or destination.\\n");
            continue;
        }

        printf("\nFinding suitable route...\\n");

        findShortestRoute(source, destination);

        printf("Delivery Status   : SUPPLY DISPATCHED\\n");
    }
}

```

```

    printf("\nAll relief requests have been processed.\n");
}

/* ----- PERFORMANCE TEST ----- */

void performanceTest() {

    int sizes[] = {10, 20, 40, 60, 80};
    int testCount = 5;

    printf("\n===== PERFORMANCE TEST =====\n");
    printf("Requests\tTime (sec)\n");

    for (int j = 0; j < testCount; j++) {

        int numberOfRequests = sizes[j];

        clock_t start = clock();

        for (int i = 0; i < numberOfRequests; i++) {

            ReliefRequest testRequest;

            testRequest.requestID = i + 1;

            strcpy(testRequest.source,
                "Central Warehouse");

            strcpy(testRequest.destination,
                "Affected Area A");

            strcpy(testRequest.supplyType,
                "Water");

            testRequest.quantity = 100 + (i % 50);

            testRequest.urgency =
                50 + (i % 51);

            insertPriorityQueue(testRequest);
        }

        while (heapSize > 0) {
            extractHighestPriority();
        }

        clock_t end = clock();
    }
}

```

```

        double executionTime =
            (double)(end - start) /
            CLOCKS_PER_SEC;

        printf("%d\t\t%.6f\n",
            numberOfRequests,
            executionTime);
    }
}

/* ----- MAIN PROGRAM ----- */

int main() {

    printf("=====\n");
    printf("    INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN\n");
    printf("    USING GRAPHS, HEAPS AND PRIORITY QUEUES\n");
    printf("=====\n");

    /* -----
    STEP 1: INITIALIZE LOCATIONS
    ----- */

    int warehouse =
        addLocation("Central Warehouse");

    int reliefCentre =
        addLocation("Relief Centre");

    int hospital =
        addLocation("Emergency Hospital");

    int areaA =
        addLocation("Affected Area A");

    int areaB =
        addLocation("Affected Area B");

    int areaC =
        addLocation("Affected Area C");

    /* -----
    STEP 2: CREATE TRANSPORTATION NETWORK
    ----- */

    addBidirectionalRoute(warehouse,
        reliefCentre,
        5);

```



```

addBidirectionalRoute(warehouse,
                      hospital,
                      8);

addBidirectionalRoute(reliefCentre,
                      areaA,
                      7);

addBidirectionalRoute(reliefCentre,
                      areaB,
                      10);

addBidirectionalRoute(hospital,
                      areaB,
                      4);

addBidirectionalRoute(hospital,
                      areaC,
                      6);

addBidirectionalRoute(areaA,
                      areaC,
                      9);

addBidirectionalRoute(areaB,
                      areaC,
                      5);

/* Display graph */
displayGraph();

/* -----
STEP 3: CREATE RELIEF REQUESTS
----- */

ReliefRequest requests[] = {

    {1,
     "Central Warehouse",
     "Affected Area A",
     "Drinking Water",
     500,
     85},

    {2,
     "Central Warehouse",
     "Affected Area B",
     "Medicines",
     200,

```

```

98},

{3,
"Central Warehouse",
"Emergency Hospital",
"Medical Kits",
150,
100},

{4,
"Central Warehouse",
"Affected Area C",
"Food Packets",
700,
75},

{5,
"Central Warehouse",
"Relief Centre",
"Blankets",
300,
65},

{6,
"Central Warehouse",
"Affected Area B",
"Drinking Water",
400,
92}
};

int requestCount =
    sizeof(requests) /
    sizeof(requests[0]);

/* -----
STEP 4: INSERT REQUESTS INTO MAX HEAP
----- */

printf("\n===== INSERTING RELIEF REQUESTS =====\n");

for (int i = 0; i < requestCount; i++) {

    printf("Request %d inserted | Destination: %s | "
        "Urgency: %d\n",
        requests[i].requestID,
        requests[i].destination,
        requests[i].urgency);

```

```

    insertPriorityQueue(requests[i]);
}

/* Display priority queue */
displayPriorityQueue();

/* -----
STEP 5: PROCESS REQUESTS
----- */

processRequests();

/* -----
STEP 6: PERFORMANCE TEST
----- */

performanceTest();

/* -----
FINAL SUMMARY
----- */

printf("\n=====\\n");
printf("        SYSTEM SUMMARY\\n");
printf("=====\\n");

printf("Locations Managed    : %d\\n",
      locationCount);

printf("Relief Requests Processed: %d\\n",
      requestCount);

printf("Graph Structure      : Weighted Adjacency List\\n");

printf("Route Algorithm      : Dijkstra's Algorithm\\n");

printf("Priority Structure    : Max Heap\\n");

printf("Priority Queue       : Urgency-Based\\n");

printf("System Status        : SUCCESS\\n");

printf("=====\\n");

return 0;
}

```

Output Screen Shots:

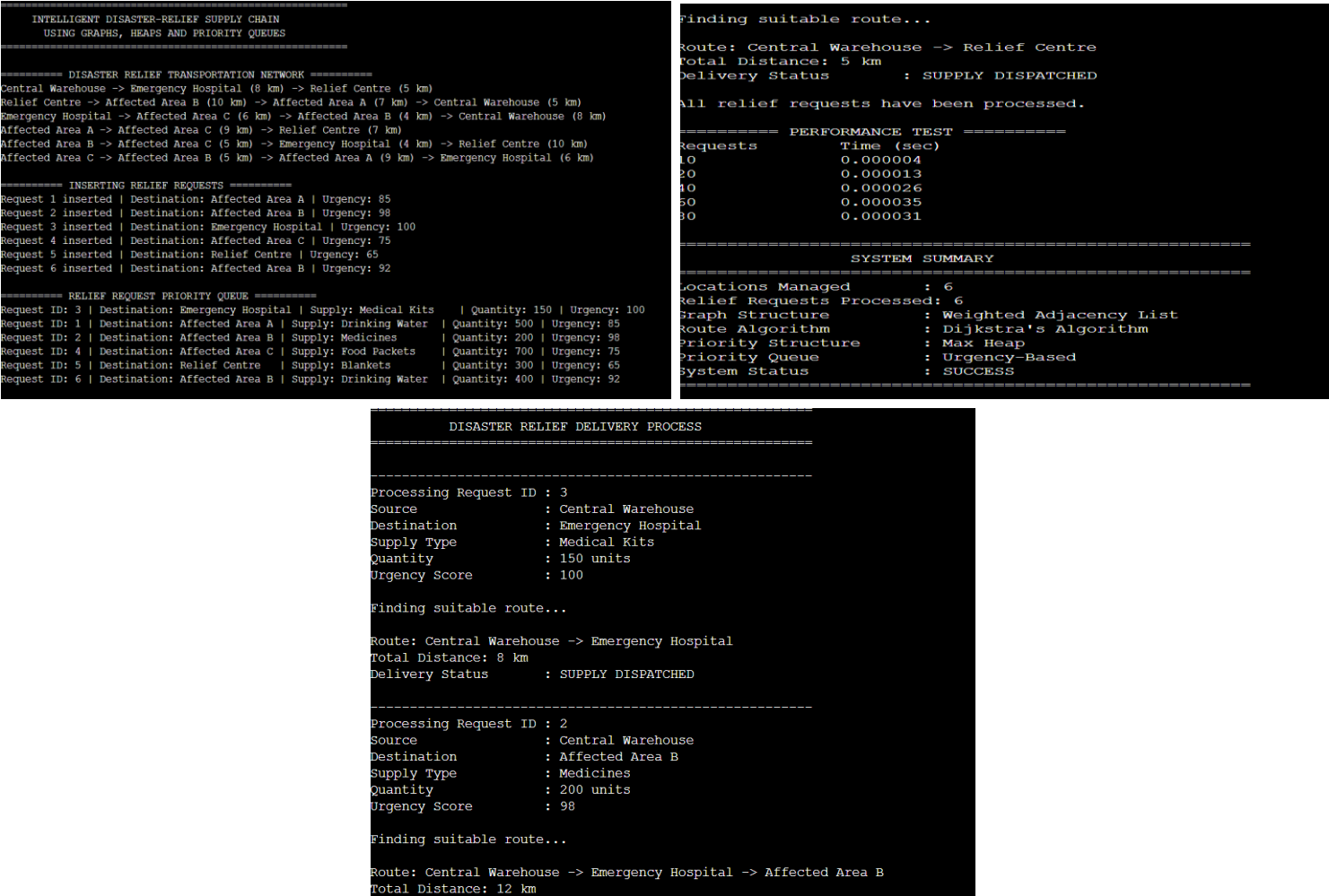


Fig 2: Output Screen Shot

Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution
clock()	Execution-time measurement

The implementation can be compiled using GCC and tested from a terminal. Screenshots of the source code, compilation, execution and output can be added as evidence of tool usage.

3. Results and Validation

Sample Input

Destination	Supply Type	20	Quantity	urgency Score
Affected Area A	Drinking Water		500	85

Affected area B	Medicines	200	100
Emergency hospital	Medical kits	150	75
Affected area C	Food packets	700	65
Relief centre	Blankets	300	92

Expected priority processing

Rank	Request ID	Destiniation	Urgency
1	3	Emergency hospital	100
2	2	Affected area b	98
3	6	Affected area b	92
4	1	Affected area a	85
5	4	Affected area c	75
6	5	Relief centre	65

Performance Test

Number of requests	Measured Time (sec)
10	Record from program
20	Record from program
40	Record from program
60	Record from program
80	Record from program

The execution-time values should be copied from the actual program output. The test demonstrates how hash insertion performance changes as the number of events increases.

4. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear route Search	Simple	Becomes inefficient for large networks
Adjacency List Graph	Efficient storage of connected locations	Route processing depends on network structure
Weighted Graph + Dijkstra	Finds shortest-distance routes between locations	Requires additional computation for route selection
Max Heap	Fast access to the most urgent relief request	Does not maintain complete sorted order
Priority Queue	Ensures critical requests are processed first	Lower-priority requests may be delayed

Hashing was selected for duplicate-IP detection because average lookup is $O(1)$. AVL was selected instead of an ordinary BST because it maintains balance and provides $O(\log n)$ operations. A max heap was selected for continuously retrieving the highest-risk event. Combining the three structures provides better functionality than relying on a single data structure.

5. Broader Considerations

- Societal Impact:

The system enables faster delivery of essential supplies to disaster-affected areas. Priority-based processing ensures that highly critical locations receive support first.

- Environmental Impact:

Shortest-route selection helps reduce unnecessary travel distance and fuel consumption. This supports more efficient and environmentally responsible relief operations.

- Ethical Considerations:

Relief requests should be prioritized using accurate and unbiased urgency scores. Supplies must be distributed fairly based on actual emergency requirements.

6. Conclusion

The proposed system efficiently manages disaster-relief supply distribution using graphs, heaps, and priority queues. The weighted graph represents transportation routes and helps identify suitable shortest paths. The max heap ensures that high-urgency relief requests are processed before lower-priority requests. The combination of these data structures improves route selection, request management, and resource utilization. Overall, the system provides a scalable approach for faster and more organized disaster-relief operations.

7. Student Reflection

Through this assignment, I learned how graphs can be used to represent real-world transportation networks.

I understood how heaps and priority queues can prioritize emergency requests based on urgency. Implementing Dijkstra's algorithm helped me understand shortest-path problems and route optimization.

I also improved my C programming, debugging, and performance-analysis skills. This project helped me understand how data structures can be combined to solve practical disaster-management problems.

8. References

- Roushan, A., Das, A., Dutta, A., & Bera, U. K., “A Multi-Objective Supply Chain Model for Disaster Relief Optimization Using Neutrosophic Programming and Blockchain-Based Smart Contracts,” *Supply Chain Analytics*, 2025.
- Passacantando, M., & Raciti, F., “A Three-Stage Game Model of the Supply Chain in Disaster Relief Operations,” *Journal of Optimization Theory and Applications*, Springer, 2025.
 - Cook, R. A., & Lodree, E. J., “Vehicle Dispatching Policies for Last Mile Distribution in a Disaster-Relief Supply Chain Network,” *Annals of Operations Research*, Springer, 2025.
 - Ahmed, H. N., Maoua, Z., Ahmed, S., & Ali, S. M., “Humanitarian Relief Supply Chain Performance Measurement: A Framework and Validation,” *Journal of Industrial Information Integration*, 2025.
 - Singh, J., Jaggi, C. K., & Kamna, K. M., “A Model for Relief Supply Chain Optimisation in a Multi-Product Distribution Environment,” *International Journal of Logistics Systems and Management*, 2025.
 - Arismawan, M., Short, M., Al-Mhdawi, M. K. S., Sentia, P. D., & Sari, C. M. A., “Humanitarian Logistics Supply Chain Management Framework: A Literature Review,” *IOP Conference Series: Earth and Environmental Science*, 2025.
- Yılmaz, Ö. F., Guan, Y., & Yılmaz, B. G., “Designing a Resilient Humanitarian Supply Chain by Considering Viability Under Uncertainty: A Machine Learning Embedded Approach,” *Transportation Research Part E: Logistics and Transportation Review*, 2025.
- Nawazish, M., & Padhi, S. S., “Designing Integrated Relief Aid Procurement and Last-Mile Distribution Strategies for Disaster Response Operations,” *Transportation Research Part E: Logistics and Transportation Review*, 2025.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem understanding & formulation	CO2	PO1, PO2	L4 – Analyze	10
Application of course/domain knowledge	CO2, CO3	PO1, PO2	L4 – Analyze	20
Solution methodology / design / implementation	CO3, CO4	PO3, PO5	L5 – Evaluate	20
Use of appropriate modern tools / techniques	CO4	PO5, PO10	L5 – Evaluate	10
Results, testing & validation	CO4, CO5	PO4, PO5	L4 – Analyze	15
Analysis, trade-offs & justification	CO3, CO5	PO2, PO4	L5 – Evaluate	15
Broader considerations / professional responsibility	CO5	PO6, PO7	L5 – Evaluate	5
Technical documentation & reflection	CO5	PO9, PO10	L5 – Evaluate	5
TOTAL	—	—	—	100

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis: Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____